

Summary

Nguyễn Hoàng Nguyễn

I. Transformer

1 Overview

Transformer là một kiến trúc mạng neural dựa hoàn toàn trên cơ chế attention, được đề xuất nhằm khắc phục các hạn chế của các mô hình tuần tự truyền thống như RNN và LSTM. Không giống các kiến trúc trước đó, Transformer xử lý toàn bộ chuỗi song song và học quan hệ phụ thuộc dài hạn một cách hiệu quả thông qua cơ chế self-attention.

2 Input Representation

2.1 Token Embedding

Chuỗi đầu vào được chia thành n token thông qua quá trình tokenization. Mỗi token sau đó được ánh xạ thành một vector embedding có chiều d_{model} thông qua một embedding layer có thể học được.

$$X \in \mathbb{R}^{n \times d_{\text{model}}}$$

Trong đó n là số lượng token trong chuỗi đầu vào và d_{model} là kích thước không gian embedding.

2.2 Positional Encoding

Do Transformer không sử dụng kiến trúc tuần tự, mô hình không tự động nắm bắt được thứ tự của các token trong chuỗi. Để khắc phục điều này, thông tin vị trí được bổ sung thông qua *positional encoding* và được cộng trực tiếp vào embedding của token:

$$Z_0 = X + \text{PE}$$

Với sinusoidal PE:

$$\begin{aligned}\text{PE}(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \\ \text{PE}(\text{pos}, 2i + 1) &= \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)\end{aligned}$$

Positional encoding cho phép mô hình phân biệt được vị trí tương đối và tuyệt đối của các token trong chuỗi.

3 Encoder

Encoder bao gồm N Encoder Blocks được xếp chồng lên nhau. Mỗi block có nhiệm vụ học biểu diễn ngữ cảnh toàn cục cho từng token trong chuỗi, đồng thời giữ nguyên độ dài chuỗi đầu vào.

4 Encoder Block

4.1 Multi-Head Self-Attention

Linear Projection. Với đầu vào Z , ba ma trận truy vấn (Query), khóa (Key) và giá trị (Value) được tạo ra thông qua các phép chiếu tuyến tính:

$$Q = ZW_Q, \quad K = ZW_K, \quad V = ZW_V$$

trong đó:

$$W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

Scaled Dot-Product Attention.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Việc chia cho $\sqrt{d_k}$ giúp ổn định gradient trong quá trình huấn luyện.

Multi-Head Attention. Để tăng khả năng biểu diễn, attention được mở rộng thành nhiều head song song:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

4.2 Residual Connection and Layer Normalization

Đầu ra của Multi-Head Attention được cộng với đầu vào ban đầu thông qua residual connection, sau đó được chuẩn hóa bằng Layer Normalization:

$$Z' = \text{LayerNorm}(Z + \text{MultiHead}(Z))$$

Residual connection giúp duy trì thông tin gốc và cải thiện khả năng huấn luyện các mô hình sâu.

4.3 Feed Forward Network

Mỗi token được xử lý độc lập bởi một mạng Feed Forward Network gồm hai lớp tuyến tính và một hàm kích hoạt ReLU:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

với:

$$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$$

4.4 Add and Layer Normalization

$$Z_{\text{out}} = \text{LayerNorm}(Z' + \text{FFN}(Z'))$$

5 Decoder

Decoder gồm N Decoder Blocks, được thiết kế để sinh token tiếp theo dựa trên các token đã được sinh trước đó và (nếu có) thông tin từ Encoder.

5.1 Masked Multi-Head Self-Attention

Khác với Encoder, Decoder sử dụng causal mask để đảm bảo rằng mỗi token chỉ có thể chú ý đến các token trước đó:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V$$

Trong đó M là ma trận mask với các giá trị $-\infty$ tại các vị trí tương ứng với token tương lai.

5.2 Encoder–Decoder Attention (Cross-Attention)

Trong lớp attention này, Query đến từ Decoder trong khi Key và Value đến từ Encoder:

$$Q = Z_{\text{decoder}}W_Q$$

$$K = Z_{\text{encoder}}W_K, \quad V = Z_{\text{encoder}}W_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Cơ chế này cho phép liên kết đầu ra Encoder với ngữ cảnh đầu vào.

5.3 Feed Forward Network

Giống với FFN trong Encoder.

6 Residual Connections

Mỗi sub-layer đều có:

$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Giúp tránh vanishing gradient và duy trì biểu diễn.

7 Output Layer and Training Objective

Đầu ra cuối cùng của Decoder được chiếu tuyến tính sang không gian từ vựng:

$$\text{logits} = ZW_{\text{vocab}}$$

Sau đó áp dụng hàm Softmax để thu được phân phối xác suất:

$$P(y_i) = \frac{e^{\text{logit}_i}}{\sum_j e^{\text{logit}_j}}$$

Mô hình được huấn luyện bằng hàm mất mát cross-entropy cho bài toán language modeling:

$$\mathcal{L} = - \sum_t \log P(y_t \mid y_{<t})$$